

Loja de Música – Relatório Técnico da 3. Iteração

Dinis Rodrigues 1250853, Guilherme Martins 1251012, Mateus Rodrigues
1251275, Rodrigo Santos 1251419
Grupo 3, Turma 1DA

Fundamentos de Desenvolvimento de Software
2025/2026
Licenciatura Engenharia Telecomunicações e Informática
Isep

Resumo. O presente relatório documenta a terceira e última iteração do desenvolvimento de uma aplicação em C++, em interface de linha de comandos, destinada à gestão do catálogo de uma loja de música. O sistema organiza um acervo de artistas e álbuns e disponibiliza funcionalidades distintas para dois perfis: o Administrador, responsável pela manutenção do catálogo, e o Cliente, que pesquisa o acervo, gere um carrinho de compras e classifica álbuns. A solução assenta numa arquitetura em camadas (Apresentação, Controlo, Domínio e Persistência), reforçada por uma camada transversal de tratamento de exceções e pela validação de invariantes de domínio. A persistência dos dados é assegurada através de ficheiros CSV, garantindo a retenção do estado entre execuções. Descrevem-se ainda os requisitos, o modelo de domínio, o diagrama de classes, a interface implementada e a estratégia de testes adotada.

Palavras-chave: Programação Orientada a Objetos · Arquitetura em Camadas · Encapsulamento · Tratamento de Exceções · Princípios GRASP · Persistência em Ficheiros · Testes Automatizados

1 Introdução

O presente relatório técnico descreve a fase de implementação e testagem de uma aplicação desenvolvida em C++ para a gestão do catálogo de uma loja de música. O sistema organiza um acervo composto por artistas, álbuns e respetivos suportes (médias), limitando o seu âmbito à integridade e à coerência do catálogo comercial, sem abranger a logística de inventário ou o controlo de stock.

Correspondendo à terceira e última iteração do ciclo de desenvolvimento, este documento consolida a evolução do software desde o levantamento de requisitos e a estabilização da interface de linha de comandos (CLI) nas fases anteriores. Na sua forma atual, a aplicação serve dois perfis de utilizador com responsabilidades distintas: o Administrador, com permissões para gerir o acervo através de operações de adição e remoção de entidades; e o Cliente, que navega no catálogo com recurso a mecanismos de filtragem e pesquisa por género, artista ou ano de lançamento, gere um carrinho de compras e classifica álbuns.

O principal desafio técnico desta iteração foi a integração coesa das quatro camadas lógicas da aplicação — Apresentação, Controlo, Domínio e Persistência — sem

comprometer o baixo acoplamento entre módulos nem a integridade dos dados em memória. Para o superar, recorreu-se a um mecanismo centralizado de tratamento de exceções, que valida as regras de negócio e impede a terminação abrupta do programa; à remoção em cascata de entidades, que evita registos órfãos; e ao mapeamento do estado do catálogo para ficheiros CSV, que assegura a persistência dos dados entre execuções.

O presente documento encontra-se organizado da seguinte forma: a Secção 2 enuncia as características e regras de integridade do sistema; a Secção 3 formaliza os requisitos através de histórias de utilizador; as Secções 4 e 5 apresentam, respetivamente, o modelo de domínio e o diagrama de classes que os traduzem numa arquitetura concreta; a Secção 6 detalha os casos de uso; a Secção 7 descreve a interface implementada para ambos os perfis; e a Secção 8 expõe a estratégia de testes unitários e de integração adotada.

2 Características do Sistema

A aplicação foi desenhada para operacionalizar o funcionamento de uma loja de música através da gestão e catalogação estruturada do seu portfólio comercial. O comportamento do sistema e o seu modelo de negócio são delimitados pelas seguintes características e regras de integridade:

- **Entidades do Domínio:** O ecossistema da aplicação é centralizado em três grandes conceitos de dados fundamentais: o **Catálogo** (entidade agregadora global), o **Artista** (autor ou intérprete) e o **Álbum** (unidade comercial que armazena os detalhes da obra e o tipo de suporte ou média).
- **Relações de Multiplicidade:** O modelo conceptual dita que a interação entre as entidades do acervo obedece a regras rígidas de cardinalidade:
 - Um **Artista** pode ter associado zero ou múltiplos álbuns no catálogo.
 - Um **Álbum** pertence obrigatoriamente a um, e apenas um, artista.
- **Perfis de Utilização:** O sistema expõe funcionalidades distintas consoante o ator que interage através da linha de comandos :
 - O **Administrador** detém a responsabilidade de gerir o acervo, possuindo privilégios para adicionar e remover registos.
 - O **Cliente** interage com o sistema para explorar o portfólio disponível, efetuar pesquisas avançadas e gerir um carrinho de compras virtual – adicionando ou removendo álbuns –, culminando na simulação da finalização da compra (checkout) com a emissão do respetivo recibo no ecrã.
- **Invariantes de Domínio (Restrições de Validação):** Para impedir a entrada de dados inconsistentes na memória RAM, o sistema valida ativamente os limites lógicos do negócio:
 - Cada artista é indexado por um identificador (ID) estritamente único.
 - Não são admitidos álbuns com preços de comercialização negativos.

- O ano de lançamento de qualquer obra musical está confinado ao intervalo cronológico entre **1900** e **2026**.
- **Consistência e Ciclo de Vida (Remoção em Cascata):** O sistema garante que não existem produtos órfãos em memória. Caso um artista seja removido do catálogo, todos os álbuns associados a esse ID são eliminados automaticamente em cascata.
- **Persistência Abstrata:** O estado do catálogo é preservado de forma não-volátil através do mapeamento direto para ficheiros de texto estruturados em formato **CSV** (artistas.csv e álbuns.csv), garantindo a retenção dos dados entre execuções do programa.

3 Histórias de Utilizador (User Stories)

Para especificar os requisitos funcionais do sistema sob a perspetiva de quem interage com a aplicação, foram definidas as seguintes histórias de utilizador, mapeadas de acordo com os perfis de acesso estruturados no sistema:

3.1 Perfil: Administrador / Gerente

O administrador interage com o sistema para efetuar a manutenção e garantir a atualização do acervo comercial da loja.

- **US01 - Inserir Artista:** Como Administrador, quero registar um novo artista no sistema (fornecendo o seu nome, país e género musical) para expandir a base de autores disponíveis no catálogo.
- **US02 - Inserir Álbum:** Como Administrador, quero adicionar um novo álbum ao catálogo e vinculá-lo a um artista existente para disponibilizar novas obras associadas aos seus respetivos autores.
- **US03 - Remover Artista em Cascata:** Como Administrador, quero remover um artista do catálogo para que o sistema elimine automaticamente o seu registo e todos os álbuns associados, evitando dados órfãos em memória.
- **US04 - Persistência de Dados (Gravação):** Como Administrador, quero salvar as alterações efetuadas no catálogo para garantir que os dados são escritos nos ficheiros CSV e preservados para utilizações futuras do programa.

3.2 Perfil: Cliente

O Cliente interage com o sistema para explorar o portfolio disponível e efetuar pesquisas avançadas. Adicionalmente, possui a capacidade de gerir um carrinho de compras virtual – adicionando ou removendo álbuns –, culminando na simulação da finalização da compra (checkout) com a emissão do respetivo recibo no ecrã:

- **US05 - Listar Catálogo Completo:** Como Cliente, quero visualizar a listagem organizada de todos os artistas e álbuns registrados no sistema para conhecer a oferta comercial disponível na loja.
- **US06 - Pesquisa e Filtragem Avançada:** Como Cliente, quero filtrar o acervo musical por critérios específicos (como género ou ano de lançamento) para localizar rapidamente as obras que correspondem às minhas preferências de consulta.
- **US07 – Gestão do Carrinho:** Como Cliente, quero adicionar ou remover álbuns de um carrinho de compras virtual para organizar as obras que pretendo adquirir numa simulação de compra.
- **US08 – Finalizar Compra (Checkout):** Como Cliente, quero finalizar a simulação de uma compra do meu carrinho para visualizar o recibo formatado com o valor total acumulado no ecrã.
- **US09 – Avaliação de Álbuns (Rating):** Como Cliente, quero atribuir uma classificação de 1 a 5 estrelas aos álbuns do catálogo para expressar a minha opinião sobre as obras disponíveis.

4 Modelo de Domínio

O Modelo de Domínio apresentado na Figura 1. Modelo de Domínio Conceptual do Sistema. ilustra a perspectiva concetual e abstrata do sistema, mapeando exclusivamente os conceitos do mundo real (entidades de negócio), os seus atributos fundamentais e a forma como interagem entre si, sem qualquer vinculação a decisões técnicas de programação ou de linguagem C++.

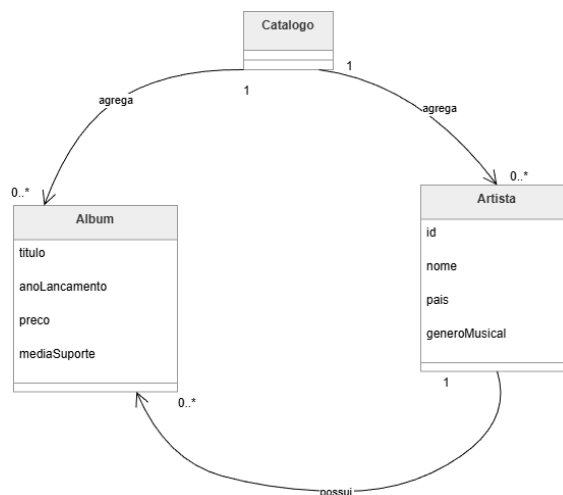


Figura 1. Modelo de Domínio Conceptual do Sistema.

4.1 Descrição das Entidades e Atributos

- **Catálogo:** Funciona como a entidade agregadora global do sistema. Não possui atributos próprios complexos, mas serve de ponto de ancoragem para a coleção de registos de loja.
- **Artista:** Representa o autor ou intérprete musical. É caracterizado pelos seguintes atributos concetuais:
 - **Identificador (ID):** Código numérico único de indexação.
 - **Nome:** Designação do artista ou banda.
 - **País:** Pátria de origem do músico.
 - **Género Musical:** Estilo predominantemente associado (Ex: Rock, Jazz, Pop).
- **Álbum:** Representa a unidade comercial da obra musical. É caracterizado por:
 - **Título:** Nome do álbum.
 - **Ano de Lançamento:** Ano cronológico da edição da obra. (do lançamento da obra)
 - **Preço:** Valor comercial de venda.
 - **Média/Suporte:** O tipo de formato físico ou digital (Ex: Vinil, CD, Cassete).

4.2 Relações e Multiplicidades

As associações entre entidades refletem as regras de negócio estipuladas para o acervo:

1. **Catálogo possui Artista/Álbum:** O *Catálogo* agrega 0..*(zero ou mais) *Artistas e Álbuns*.
2. **Artista compõe Álbum:** A ligação core entre o *Artista* e o *Álbum* estabelece uma relação de 1 para muitos:
 - Um **Artista** pode ter associado 0..*(Zero ou mais) *Álbuns* no sistema.
 - Um **Álbum** está estritamente vinculado a exatamente 1 *Artista* (multiplicidade unitária), impossibilitando a existência de uma obra sem autor atribuído.

5 Diagrama de Classes

O Diagrama de Classes constitui um pilar fundamental na engenharia de software, servindo como a ponte técnica que traduz o Modelo de Domínio abstrato numa arquitetura concreta, estruturada e executável na linguagem C++. Ao mapear não apenas as entidades do negócio, mas também os seus atributos tipificados, métodos, modificadores de acesso e mecanismos de encapsulamento, este diagrama oferece uma perspetiva estática rigorosa de como o software está efetivamente construído na memória RAM.

Para garantir uma análise detalhada e progressiva da aplicação desenvolvida, esta secção divide-se em duas perspetivas complementares:

1. **Diagrama de Classes do Domínio (Secção 5.1):** Uma visão focada exclusivamente nas entidades core e na lógica pura do negócio (Modelos Ricos), omitindo detalhes de infraestrutura ou interface.
2. **Diagrama Arquitetura Completo (Secção 5.2):** Uma visão global do sistema que ilustra a interação e dependência entre as **quatro** camadas lógicas da aplicação – Apresentação (Views), Controlo (Controllers), Domínio (Models) e Persistência (Persistence) -, bem como o mapeamento da camada transversal de tratamento de erros e Exceções (Exceptions). Esta abordagem evidencia o respeito pelos princípios GRASP de baixo acoplamento e alta coesão, demonstrando como o sistema captura e mitiga falhas em tempo de execução através de exceções customizadas.

5.1 Diagrama de Classes do Domínio:

O Diagrama de Classes do Domínio, ilustrado na Figura 2, detalha a especificação técnica e o encapsulamento das entidades centrais que operam em memória RAM: Artista, Álbum e Carrinho. Diferenciando-se do cariz puramente abstrato do Modelo de Domínio (Secção 4), esta perspetiva técnica introduz formalmente a tipificação de dados nativa do C++ e os modificadores de acesso que protegem a integridade das propriedades de cada objeto.

Ao omitir por completo as classes de infraestrutura, os mecanismos de persistência em disco ou as rotinas lógicas da interface de utilizador (CLI), este diagrama oferece uma visão perfeitamente limpa e coesa:

- **Encapsulamento e Invariantes:** As classes Artista e Álbum protegem os seus atributos através de visibilidade privada (-), forçando a validação de regras de negócio (como a barreira cronológica de anos ou preços não-negativos) diretamente no ato de **instanciação**.
- **A Inclusão do Carrinho:** O Carrinho surge mapeado como uma entidade do domínio autónoma, agregando referências ou instâncias dinâmicas de Álbum, o que viabiliza a simulação lógica de compras sem sobrecarregar a estrutura estática do catálogo comercial.

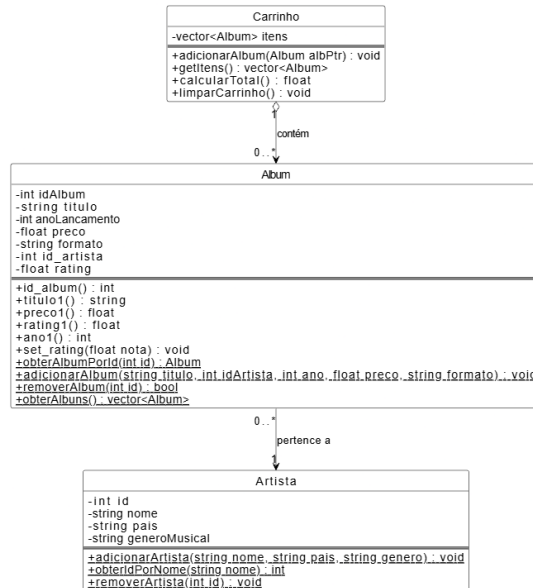
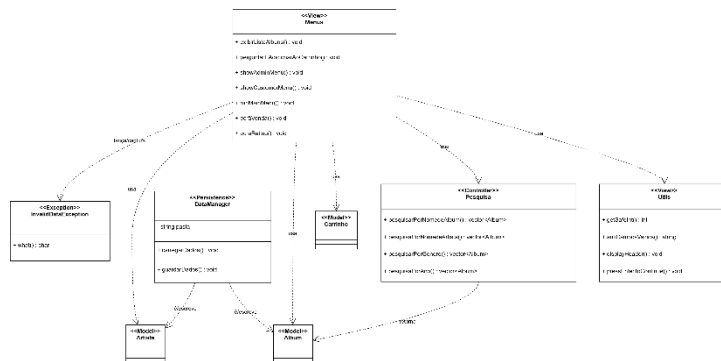


Figura 2. Diagrama de Classes de Domínio da Aplicação.

5.2 Diagrama Arquitetural Completo:

A

Figura 3 mapeia a divisão do sistema em quatro camadas lógicas: Apresentação, Controlo, Domínio e Persistência. As setas de dependência demonstram a aplicação prática dos princípios GRASP (baixo acoplamento), evidenciando o fluxo seguro de informação desde a interação do utilizador até à gravação dos dados nos ficheiros CSV.



6 Casos de Uso (Use Cases)

A interação com o sistema está mapeada através dos seguintes Casos de Uso, divididos pelos dois atores principais do sistema:

6.1 Administrador (Gestão do Catálogo)

- **UC01 - Inserir Artista:** Registrar um novo autor no sistema, fornecendo nome, país e gênero musical.
- **UC02 - Inserir Álbum:** Adicionar uma nova obra comercial e vinculá-la obrigatoriamente a um artista existente.
- **UC03 - Remover Artista em Cascata:** Eliminar um artista e garantir que todos os álbuns a ele associados são apagados automaticamente, evitando referências órfãs na memória.
- **UC04 - Remover Álbum:** Eliminar um álbum específico do catálogo através da pesquisa pelo seu nome e seleção do seu ID exato, sem afetar o artista.
- **UC05 - Persistência de Dados:** Efetuar a leitura e escrita automática do estado do catálogo nos ficheiros CSV durante o arranque e encerramento da aplicação.
- **UC06 - Autenticação Administrativa:** Validar o acesso à área de gestão através de uma palavra-passe, com captura de exceções para tentativas inválidas.

6.2 Cliente (Exploração e Compra)

- **UC07 - Listar Catálogo Completo:** Visualizar a oferta total da loja, com a listagem formatada de álbuns, preços e notas de avaliação.
- **UC08 - Pesquisa e Filtragem Avançada:** Interrogar o catálogo através de filtros específicos como o Título do Álbum, Nome do Artista, Género Musical ou Ano de Lançamento.
- **UC09 - Gestão do Carrinho:** Adicionar álbuns ao carrinho temporário, calcular o valor total acumulado e esvaziar a lista de compras.
- **UC10 - Finalizar Compra (Checkout):** Confirmar os artigos do carrinho, simular o pagamento e gerar um recibo de venda no ecrã.
- **UC11 - Avaliação de Álbuns (Rating):** Permitir ao cliente pesquisar um álbum específico e atribuir-lhe uma classificação de qualidade numa escala de 1 a 5 estrelas.

7 Interface de Utilizador / Administrador (User / Manager Interface)

A interface da aplicação, materializada através de uma Linha de Comandos (CLI), foi projetada sob o princípio de separação de responsabilidades (camada de *Views* e utilitários associados), garantindo uma interação estruturada entre os atores do sistema e a lógica de negócio. O ecossistema da interface encontra-se logicamente particionado em duas áreas de atuação mutuamente exclusivas, mapeadas de acordo com os perfis de acesso previamente definidos: a vertente de manutenção do catálogo (Administrador) e a vertente de exploração comercial (Cliente).

7.1 Arquitetura da Interface

A navegação assenta numa estrutura de menus hierárquicos controlados de forma centralizada. Para preservar a integridade do sistema em tempo de execução, a interface atua como a primeira barreira de validação dos limites lógicos do negócio. Entradas (*inputs*) inconsistentes ou que violem as regras do domínio (como tipos de dados incorretos ou opções inexistentes) são ativamente mitigadas. O sistema captura estas falhas através do mapeamento de exceções customizadas (ex: *InvalidDataException*),

traduzindo-as em *feedback* legível no ecrã e evitando a interrupção abrupta do ciclo de vida da aplicação. A disposição estrutural desta árvore de navegação e as opções iniciais apresentadas ao utilizador encontram-se ilustradas na Figura 4.

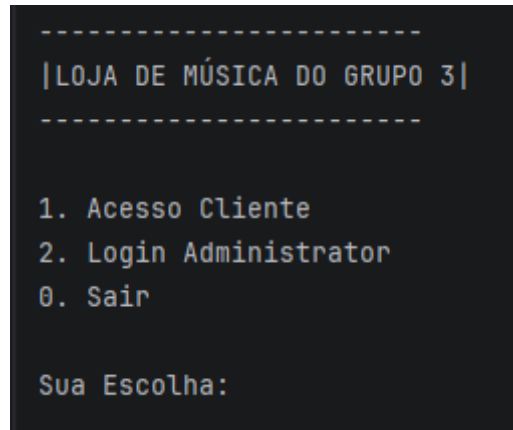


Figura 4. Menu principal de entrada da aplicação com a bifurcação dos perfis de acesso.

7.2 Perfil: Administrador (Manager Interface)

O acesso ao ambiente de gestão encontra-se estritamente delimitado por um mecanismo de autenticação administrativa (UC06), bloqueando interações não autorizadas com o portfólio da loja. Uma vez validado, a interface expõe um painel de controlo focado na integridade operacional:

- **Manutenção do Acervo:** Menus dedicados à inserção e eliminação de entidades (Artistas e Álbuns), refletindo em tempo real e de forma visual as invariantes do domínio (como a notificação de remoções automáticas em cascata).
- **Feedback de Persistência:** A interface fornece *feedback* visual sistemático sobre as operações de entrada/saída (I/O), confirmando a gravação correta do estado do catálogo em disco (ficheiros CSV) durante as transições críticas do programa. As operações exclusivas de escrita, modificação do acervo e os respetivos alertas visuais de confirmação da consola podem ser observados na Figura 5.

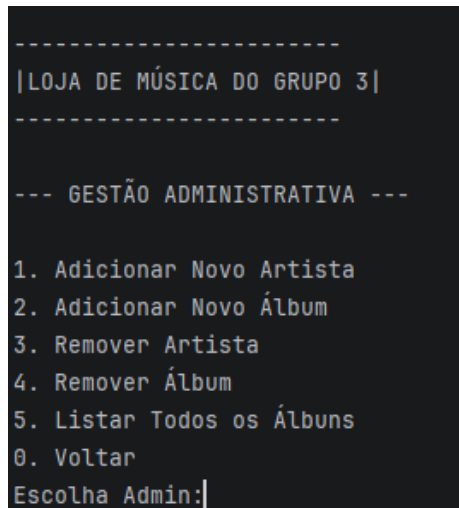


Figura 5. Painel de gestão do Administrador evidenciando as rotinas de manutenção do acervo comercial.

7.3 Perfil: Cliente (User Interface)

A interface direcionada ao ator "Cliente" abstrai a complexidade arquitetural do domínio, priorizando a navegação dinâmica e a simulação das lógicas de negócio:

- **Exploração e Filtragem:** Apresentação rigorosamente formatada e paginada do catálogo completo, conforme exemplificado na Figura 6, suportada por menus de pesquisa avançada (UC08) que permitem interrogar o acervo de acordo com restrições específicas (gênero musical, ano de lançamento ou artista).
- **Simulação de Negócio (Carrinho e *Checkout*):** A interface renderiza o estado da memória RAM referente ao carrinho de compras virtual do utilizador. O fluxo culmina na simulação de compra (UC10), gerando em ecrã a emissão gráfica de um recibo formatado, que detalha as obras seleccionadas e o valor comercial total acumulado.
- **Avaliação do Acervo:** Ecrãs interativos dedicados à submissão de *ratings* de 1 a 5 estrelas (UC11), promovendo uma interação bidirecional entre o consumidor e as obras musicais disponíveis.

```
-----
|LOJA DE MÚSICA DO GRUPO 3|
-----

--- LOJA DE CLIENTES ---

1. Pesquisar no Catálogo
2. Ver o meu Carrinho e Total
3. Esvaziar Carrinho
4. Finalizar Compra
5. Deixar Opinião sobre um Álbum
0. Voltar ao Menu Principal

Sua Escolha:
```

Figura 6. Área do Cliente demonstrando a listagem estruturada, paginação e o alinhamento de dados do catálogo.

8 Testagem (Unitária e Integração)

Para assegurar a estabilidade, a integridade das regras de negócio e a conformidade da aplicação com as histórias de utilizador mapeadas, foi desenhada e implementada uma infraestrutura completa de testes automatizados com base na framework industrial GoogleTest (gtest). A estratégia de testes abrangeu tanto a validação isolada das propriedades e comportamentos das entidades (Testes Unitários) como um fluxo transacional de comunicação entre as componentes das quatro camadas do sistema (Testes de Integração).

8.1 Infraestrutura e Arquitetura Modular de Testes

Afastando-se de uma abordagem de ficheiro único monolítico, que prejudica a manutenibilidade e a escalabilidade do código, o grupo reestruturou a suite de testes numa arquitetura estritamente modular. Esta organização mimetiza os padrões recomendados de engenharia de software, segmentando o ambiente de qualidade em cinco ficheiros de responsabilidade definida:

1. **main_test.cpp (Orquestrador Global):** Atua como o ponto de entrada central do executável de testes (**runTests**). O seu escopo limita-se a inicializar o ambiente de execução do framework através da rotina **::testing::InitGoogleTest()** e disparar o motor de asserções com **RUN_ALL_TESTS()**. Para otimizar o processo de vinculação e evitar erros

críticos de compilação por símbolos duplicados no Linker (Multiple Definition Errors), este ficheiro recorre à inclusão literal (via diretivas **#include**) das unidades de teste individuais consolidando-as numa única unidade de tradução conceptual.

2. **testes_artistas.cpp**: Módulo especializado na validação dos estados e invariantes associados à classe **Artista**.
3. **testes_album.cpp**: Módulo encarregue de auditar o comportamento da entidade **Album**, focando-se nas restrições de formatação, preço e limites cronológicos.
4. **testes_carrinho.cpp**: Módulo isolado que implementa uma *Test Fixture* dedicada para isolar o ciclo de vida e o estado transacional do carrinho virtual de compras.
5. **testes_integracao.cpp**: Módulo complexo focado em validar o acoplamento seguro entre os controladores de busca, o gestor de persistência em disco e os fluxos de checkout e feedback do sistema.

8.2 Metodologia AAA (Arrange-Act-Assert) e Especificações dos Módulos

Todos os cenários de teste foram desenvolvidos sob a disciplina do padrão AAA (**Arrange-Act-Assert**), organizando cronologicamente cada bloco de código para instanciar os pré-requisitos (*Arrange*), executar o comportamento sob análise (*Act*) e auditar o resultado face às expectativas teóricas (*Assert*).

- **Módulo de Artistas (testes_artistas.cpp)**: O cenário **CriarArtistaValido** assegura que dados conformes passam pela validação sem disparar exceções (**EXPECT_NO_THROW**), garantindo adicionalmente que a indexação na tabela de símbolos funciona através do método **obterIdPorNome**, cujo retorno deve ser obrigatoriamente um ID válido (**EXPECT_NE(id, -1)**). O cenário **BloquearCamposVazios** valida a robustez do construtor, assegurando que a introdução de strings nulas ou vazias força o imediato disparo de uma exceção de barreira do tipo **InvalidDataException** (**EXPECT_THROW**).
- **Módulo de Álbuns (testes_album.cpp)**: Este bloco blinda o domínio contra corrupção de dados na memória RAM. O cenário **ValidarPrecoNegativo** intercepta tentativas de atribuição de valores comerciais abaixo de zero. O cenário **ValidarFormatos** afere a conformidade dos suportes físicos, aceitando capitalizações higienizadas (como a conversão ativa de “vinil” para “Vinil”) e rejeitando formatos comerciais não suportados (como “Cassete”), despoletando a respetiva exceção de dados. Por fim, o cenário **ValidarLimitesDoAno** estabelece as fronteiras cronológicas fixadas nos requisitos (1900 a 2026), garantindo que os anos de fronteira inválidos (como 1899 e 2027) são ativamente bloqueados pelo sistema.
- **Módulo do Carrinho (testes_carrinho.cpp)**: Utiliza a classe especializada **CarrinhoFixture**, derivada de **::testing::Test**, para criar um arnês de testes limpos. Através do método nativo **SetUp()**, o ambiente configura instâncias limpas de ponteiros para objetos da Heap (**Artista*** e **Album***). O teste **ValidarOperacoesECalculoTotal** simula de forma encadeada adições de

itens, valida se o somatório aritmético dos preços flutuantes coincide com o esperado através de `EXPECT_FLOAT_EQ`, e afere se a remoção seletiva e a rotina de limpeza integral (`limparCarrinho`) manipulam corretamente o tamanho do vetor dinâmico de itens.

- **Módulo de Integração (testes_integracao.cpp):** Valida a cooperação multicamada do sistema. A integração com a camada de persistência é testada instanciando o `DataManager` apontando para a diretoria padrão **“data”**, aferindo que o método `carregarDados()` realiza a hidratação da memória a partir dos ficheiros CSV sem falhas. A integração com a camada de controlo avalia as funções estáticas de consulta da classe **Pesquisa** (`pesquisarPorGenero` e `pesquisaPorNomeDeArtista`). O fluxo encerra com a validação das regras efetivas de negócio: comprova-se que a entidade de feedback **Rating** armazena e calcula as pontuações na escala correta e que o objeto transacional **Venda** acopla perfeitamente um carrinho de data, um ID e uma referência direta a um objeto **Carrinho** populado, emitindo o estado final de checkout com sucesso.

8.3 Análise de Resultados e Cobertura de Código (Code Coverage)

A execução laboratorial da suite de testes através do motor de análise dinâmico do IDE Clion revelou indicadores de máxima excelência técnica:

- **Sucesso Lógico:** O sistema obteve aprovação absoluta em todos os cenários delineados, culminando em 9 testes passados com sucesso (**9 tests passed**). A execução registou uma latência virtual nula de 0 ms/ 1 ms, encerrando com o código de integridade limpo (**Process finished with exit code 0**), o que atesta a correta calibração e leveza computacional do código de verificação desenvolvido.
- **Cobertura de Linhas (Line Coverage):** O mapeamento dinâmico registou a marca ideal de **100% de Line Coverage** de forma homogênea em todos os módulos de teste (`main_test.cpp`, `testes_album.cpp`, `testes_artistas.cpp`, `testes_carrinho.cpp` e `testes_integracao.cpp`). Esta métrica matemática constitui a prova irrefutável de que a totalidade das linhas de instrução, caminhos felizes e bloco de captura de exceções foram varridos e validados de ponta a ponta, eliminando código morto ou fluxos lógicos imprevisíveis.
- **Cobertura de Ramos (Branch Coverage):** O ficheiro maestro `main_test.cpp` obteve 100% de Branch Coverage. Por sua vez, os restantes módulos de testes registaram índices consolidados na ordem dos 45% a 48% de Branch Coverage. Esta divergência percentual constitui um comportamento perfeitamente normal, standard e esperado no desenvolvimento em linguagem C++ recorrendo ao ecossistema GoogleTest. As macros de asserção nativas do framework (tais como `EXPECT_THROW` ou `EXPECT_FLOAT_EQ`) expandem-se, em fase de pré-compilação, em múltiplos ramos condicionais invisíveis gerados automaticamente pelo compilador para controlo interno de relatórios e interação de falhas. Como o `main_test.cpp` apenas efetua a inclusão de ficheiros sem declarar macros de asserção, permanece imune a este decréscimo artificial, ao passo que os módulos acusam esta métrica

devido à densidade de validação injetadas pelo gtest, não significando de forma alguma a existência de ramificações de utilizador por testar.

A validação empírica destes indicadores lógicos, evidenciando tanto a aprovação unânime dos cenários de teste como o mapeamento analítico da cobertura ideal alcançada no IDE Clion, encontra-se documentada na Figura 7.

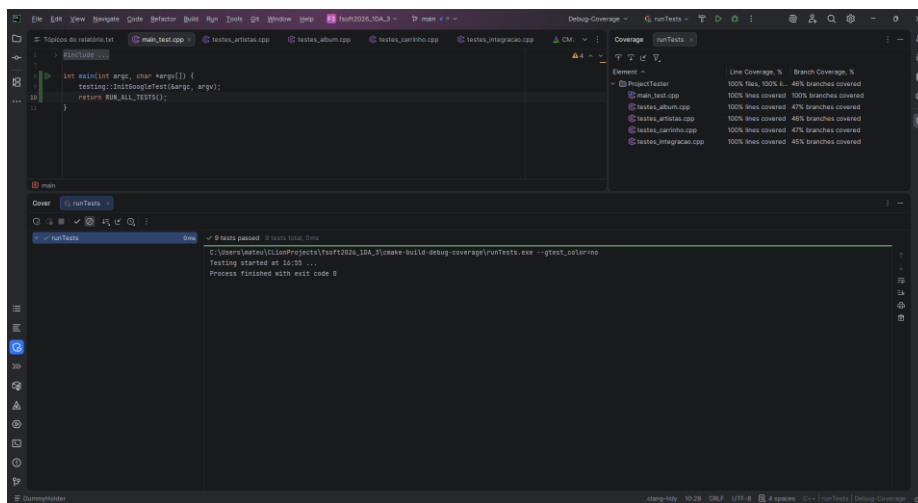


Figura 7. Painel de resultados do GoogleTest no CLion demonstrando os 9 testes aprovados e o índice de 100% de Line Coverage nos módulos de teste.

8.4 Estratégia de Gestão Dinâmica de Memória (Mitigação de Leaks)

Atendendo a que a linguagem C++ não dispõe de um mecanismo nativo de recolha de lixo (Garbage Collector) para objetos alocados dinamicamente na Heap, o grupo desenhou a arquitetura de testes com preocupações rigorosas de eficiência e estabilidade de infraestrutura, mitigando a ocorrência de fugas de memória (Memory Leaks).

Esta salvaguarda foi materializada de forma determinística na classe **CarrinhoFixture** através do ciclo da vida regulado pelo próprio GoogleTest. Sempre que um cenário de teste baseado na fixture encerra a sua exceção (quer passe com sucesso ou falhe), o framework invoca obrigatoriamente a rotina de destruição controlada **TearDown()**. Este método atua revertendo cirurgicamente as operações de escrita na Heap configuradas no **SetUp()**, aplicando o operador **delete** em cada um dos ponteiros de teste partilhados (**delete alb1;**, **delete alb2;**, **delete artistaFake;**).

Este procedimento limpa integralmente o espaço de paginação alocado ao processo na memória RAM, garantindo que execuções em larga escala ou rotinas contínuas de integração de qualidade de software rodam em ambiente estéril e seguro, sem degradação de recursos ou acumulação de lixo digital na máquina de destino.